

Dataset sintético para regresión logística (5000 registros)

A partir del ejemplo del documento —donde la compra depende de si el visitante pasa más de 5 minutos y añade más de 2 productos—, aquí tienes el código completo para generar el dataset:

```
In [5]: import numpy as np
import pandas as pd

# Semilla para reproducibilidad
np.random.seed(42)

# Número de muestras
n_samples = 5000

# Generar tiempo en sitio (minutos) entre 1 y 10
tiempo = np.random.uniform(1, 10, n_samples)

# Generar cantidad de productos en carrito entre 0 y 5
productos = np.random.randint(0, 6, n_samples)

# Regla para determinar compra:
# Si tiempo > 5 y productos > 2 → compra = 1, si no → 0
compra = ((tiempo > 5) & (productos > 2)).astype(int)

# Crear DataFrame
data = pd.DataFrame({
    'tiempo': tiempo,
    'productos': productos,
    'compra': compra
})

print(data.head())
print("\nTamaño del dataset:", data.shape)
```

	tiempo	productos	compra
0	4.370861	2	0
1	9.556429	2	0
2	7.587945	5	1
3	6.387926	3	1
4	2.404168	2	0

Tamaño del dataset: (5000, 3)

Preparación del entorno en Python

Librerías para manejo de datos

```
In [6]: import pandas as pd
import numpy as np

# Librerías para crear el modelo de regresión logística
from sklearn import linear_model, model_selection
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Librerías para visualización
import matplotlib.pyplot as plt
import seaborn as sb

# Configuración para mostrar gráficos dentro del notebook
%matplotlib inline

# Comentario:
# Este bloque prepara todas las herramientas necesarias para:
# - Cargar y manipular datos (pandas, numpy)
# - Entrenar un modelo de regresión logística (sklearn)
# - Evaluar el modelo (accuracy, matriz de confusión, reporte de clasificación)
# - Visualizar datos y resultados (matplotlib, seaborn)
```

1. Carga del dataset generado

Antes de entrenar el modelo, necesitamos cargar el dataset que generaste previamente.

En este ejemplo, asumiré que ya lo tienes en un DataFrame llamado `data`.

Si lo guardaste en un CSV, también te dejo la línea para cargarlo.

```
In [3]: # Si ya tienes el DataFrame 'data' generado en memoria, no necesitas hacer nada.  
# Si lo guardaste en un archivo CSV, descomenta la siguiente línea:  
# data = pd.read_csv("dataset_tienda_online.csv")  
  
# Mostrar primeras filas  
data.head()
```

```
Out[3]:
```

	tiempo	productos	compra
0	4.370861	2	0
1	9.556429	2	0
2	7.587945	5	1
3	6.387926	3	1
4	2.404168	2	0

2. División Train/Test

Dividimos los datos en entrenamiento (80%) y prueba (20%).

Esto permite evaluar el modelo con datos que nunca ha visto.

```
In [7]: # DIVISIÓN TRAIN/TEST  
X = data[['tiempo', 'productos']] # Variables predictoras  
y = data['compra'] # Variable objetivo  
  
# División 80% entrenamiento, 20% prueba  
X_train, X_test, y_train, y_test = model_selection.train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
X_train.shape, X_test.shape
```

```
Out[7]: ((4000, 2), (1000, 2))
```

¿Qué está haciendo este bloque de código?

1. Selección de variables

- **X** contiene las variables predictoras: `tiempo` y `productos`.
- **y** contiene la variable objetivo: `compra`.

2. División del dataset

Se usa `train_test_split` para separar los datos en:

- **80% entrenamiento** → para que el modelo aprenda.
- **20% prueba** → para evaluar el rendimiento con datos nunca vistos.

El parámetro `random_state=42` asegura que la división sea reproducible.

3. Resultado

El output:

```
((4000, 2), (1000, 2))
```

significa:

- `X_train` tiene **4000 filas** y **2 columnas**.
- `X_test` tiene **1000 filas** y **2 columnas**.

Esto cuadra perfectamente con un dataset total de 5000 filas.

3. Entrenamiento del modelo de regresión logística

Creamos el modelo, lo entrenamos y mostramos los coeficientes aprendidos.

```
In [8]: # ENTRENAMIENTO DEL MODELO

modelo = linear_model.LogisticRegression()
modelo.fit(X_train, y_train)

print("Coeficientes del modelo:", modelo.coef_)
print("Intercept:", modelo.intercept_)
```

Coeficientes del modelo: [[1.35440102 2.35022554]]
Intercept: [-16.86605224]

```
In [9]: # Código Python para representar la ecuación del modelo de regresión logística

import numpy as np

# Coeficientes del modelo ya entrenado
coef = modelo.coef_[0]      # [B1, B2]
intercept = modelo.intercept_[0] # B0

# Nombres de las variables
variables = X_train.columns

# Construcción de la ecuación logística
ecuacion = f"logit(p) = {intercept:.4f}"
for b, var in zip(coef, variables):
    ecuacion += f" + ({b:.4f})·{var}"

print("Ecuación del modelo logístico:")
print(ecuacion)

# También puedes imprimir la ecuación en forma de probabilidad:
print("\nForma probabilística:")
print("p = 1 / (1 + exp(-(B0 + B1·x1 + B2·x2)))")
```

Ecuación del modelo logístico:

$$\text{logit}(p) = -16.8661 + (1.3544) \cdot \text{tiempo} + (2.3502) \cdot \text{productos}$$

Forma probabilística:

$$p = 1 / (1 + \exp(-(B_0 + B_1 \cdot x_1 + B_2 \cdot x_2)))$$

Explicación en Markdown: ¿Qué significa el intercepto en regresión logística?

¿Qué es el intercepto?

En la ecuación logística:

$$\ln\left(\frac{p}{1-p}\right) = B_0 + B_1x_1 + B_2x_2$$

el **intercepto**

$$(B_0)$$

representa el valor del *logit* (log-odds) cuando todas las variables predictoras valen 0.

¿Qué significa que el intercepto sea negativo?

Si

$$(B_0 < 0)$$

, entonces:

$$\ln\left(\frac{p}{1-p}\right) < 0$$

lo cual implica:

$$\frac{p}{1-p} < 1 \Rightarrow p < 0.5$$

Es decir:

Un intercepto negativo significa que, cuando todas las variables predictoras valen 0, la probabilidad base del evento (p) es menor que 0.5.

En tu caso (tiempo = 0 minutos, productos = 0), el modelo considera que **la probabilidad de compra es muy baja**, lo cual es totalmente lógico:

si un usuario no pasa tiempo en la web y no añade productos, lo normal es que *no compre*.

Interpretación intuitiva

- El intercepto es la “probabilidad base” antes de considerar ninguna variable.
- Un intercepto negativo indica que el evento es improbable por defecto.
- Los coeficientes de las variables son los que “empujan” esa probabilidad hacia arriba cuando tiempo y productos aumentan.

```
In [10]: # Código para visualizar la curva Logística
import numpy as np
import matplotlib.pyplot as plt

# Coeficientes del modelo entrenado
B0 = modelo.intercept_[0]
B1, B2 = modelo.coef_[0]

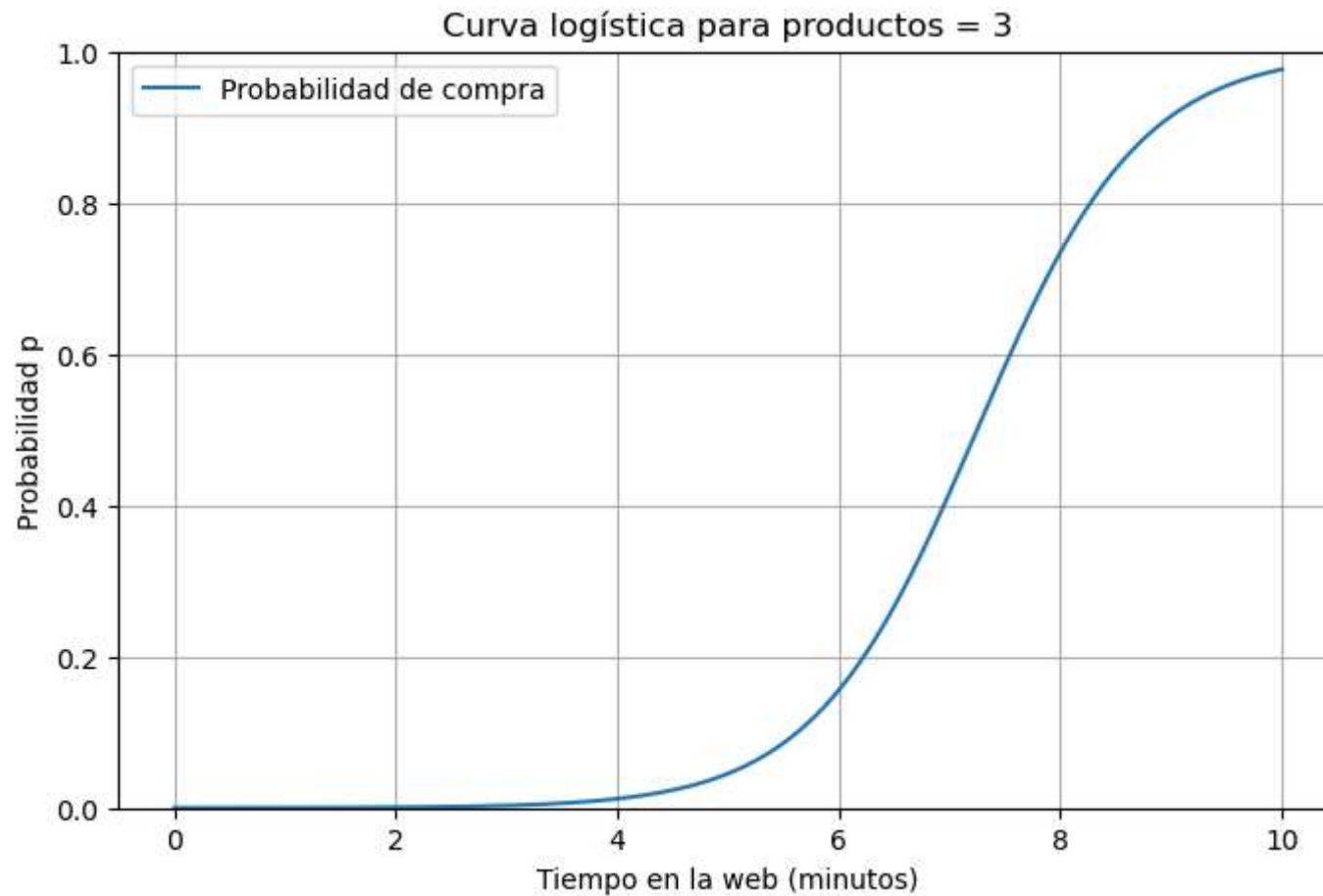
# Rango de valores para la variable "tiempo"
tiempo_vals = np.linspace(0, 10, 300)

# Fijamos productos = 3 (por ejemplo)
productos_fijo = 3

# Ecuación Logística
logit = B0 + B1 * tiempo_vals + B2 * productos_fijo
```

```
p = 1 / (1 + np.exp(-logit))

# Gráfico
plt.figure(figsize=(8,5))
plt.plot(tiempo_vals, p, label="Probabilidad de compra")
plt.xlabel("Tiempo en la web (minutos)")
plt.ylabel("Probabilidad p")
plt.title("Curva logística para productos = 3")
plt.grid(True)
plt.ylim(0, 1)
plt.legend()
plt.show()
```



¿Qué muestra este gráfico?

- La curva empieza cerca de **0**, porque con poco tiempo en la web la probabilidad de compra es baja.
- A medida que aumenta el tiempo, la probabilidad sube suavemente.
- La forma en **S** es la típica de la función logística que aparece en tu documento:

“La función logística transforma cualquier entrada real en un valor entre 0 y 1.”

- El punto donde la curva se vuelve más empinada depende de los coeficientes del modelo.

4. Evaluación completa del modelo

Incluye:

- Predicciones
- Matriz de confusión
- Accuracy
- Reporte de clasificación

```
In [11]: # EVALUACIÓN COMPLETA DEL MODELO

# Predicciones
y_pred = modelo.predict(X_test)

# Métricas
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nMatriz de confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de clasificación:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.909

Matriz de confusión:

```
[[718 40]
 [ 51 191]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.93	0.95	0.94	758
1	0.83	0.79	0.81	242
accuracy			0.91	1000
macro avg	0.88	0.87	0.87	1000
weighted avg	0.91	0.91	0.91	1000

Interpretación completa del modelo de regresión logística

1. Matriz de confusión (tabla completa)

Según la teoría de tu documento:

“La matriz de confusión muestra cómo se distribuyen los aciertos y errores...
TN, FP, FN, TP.”

La matriz que obtuviste:

```
[[718 40]
 [ 51 191]]
```

La representamos en tabla:

	Predicción 0	Predicción 1
Valor real 0	TN = 718	FP = 40
Valor real 1	FN = 51	TP = 191

2. Interpretación de cada valor

✓ TN (718)

Casos donde el modelo dijo *NO compra* y realmente era *NO compra*.

→ Aciertos correctos de clase negativa.

✓ TP (191)

Casos donde el modelo dijo *Sí compra* y realmente era *Sí compra*.

→ Aciertos correctos de clase positiva.

✗ FP (40)

El modelo predijo *compra* pero en realidad era *NO compra*.

→ En un contexto real, serían usuarios que el modelo cree que comprarán... pero no lo hacen.

✗ FN (51)

El modelo predijo *NO compra* pero en realidad era *Sí compra*.

→ Usuarios que sí iban a comprar, pero el modelo no los detectó.

3. Interpretación del Accuracy

“El accuracy es el porcentaje de predicciones correctas.”

Tu valor:

$$\text{Accuracy} = 0.909$$

Esto significa:

- El modelo acierta **el 90.9% de las veces**.
- Es un valor alto, pero recuerda lo que dice tu documento:

“No siempre es suficiente... no dice qué tipo de errores comete el modelo.”

Por eso es importante mirar la matriz de confusión y el reporte de clasificación.

4. Interpretación del Reporte de Clasificación

El reporte muestra **precision**, **recall** y **f1-score**, tal como se explica en tu PDF:

“Precision: de todas las veces que el modelo dijo 1, cuántas eran realmente 1.”

“Recall: de todos los casos que eran realmente 1, cuántos detectó el modelo.”

“F1-score: equilibrio entre precision y recall.”

Clase 0 (NO compra)

- **Precision = 0.93**
Cuando el modelo dice “NO compra”, acierta el 93% de las veces.
- **Recall = 0.95**
Detecta correctamente el 95% de todos los NO compradores.

- **F1 = 0.94**
Excelente equilibrio.

El modelo es **muy bueno** identificando a quienes NO compran.

Clase 1 (Sí compra)

- **Precision = 0.83**
Cuando predice "compra", acierta el 83% de las veces.
- **Recall = 0.79**
Detecta el 79% de los compradores reales.
- **F1 = 0.81**
Buen rendimiento, aunque menor que la clase 0.

El modelo funciona bien, pero **se le escapan compradores reales** (FN = 51).

5. Conclusión global

- El modelo es **muy sólido**, con un accuracy del 90.9%.
- Identifica muy bien a los NO compradores (clase 0).
- Identifica bien, aunque no tan bien, a los compradores (clase 1).
- Los errores más relevantes son:
 - **40 FP** → cree que comprarán, pero no lo hacen.
 - **51 FN** → compradores reales que no detecta.

En un contexto de marketing, los **FN** suelen ser más costosos, porque son ventas perdidas.

5. Visualización de la frontera de decisión

Esto permite ver cómo el modelo separa las clases en el plano (tiempo vs productos). Es muy útil para interpretar modelos simples.

```
In [12]: # VISUALIZACIÓN DE LA FRONTERA DE DECISIÓN

# Crear una malla de puntos
x_min, x_max = X['tiempo'].min() - 1, X['tiempo'].max() + 1
y_min, y_max = X['productos'].min() - 1, X['productos'].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

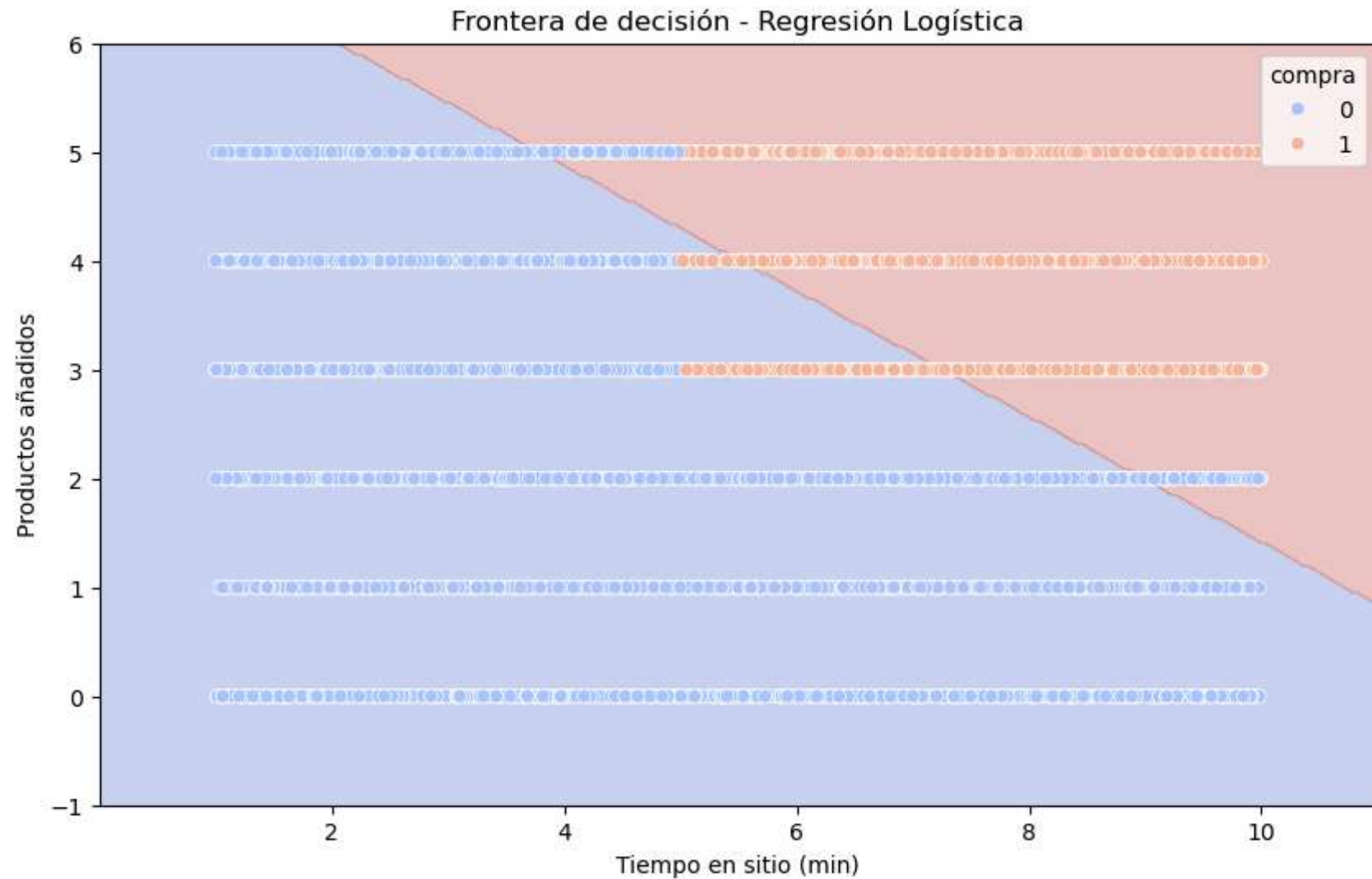
# Crear DataFrame con nombres de columnas para evitar el warning
grid = pd.DataFrame({
    'tiempo': xx.ravel(),
    'productos': yy.ravel()
})

# Predicción sobre la malla
Z = modelo.predict(grid)
Z = Z.reshape(xx.shape)

# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

# Puntos reales
sb.scatterplot(data=data, x='tiempo', y='productos', hue='compra', palette='coolwarm')

plt.title("Frontera de decisión - Regresión Logística")
plt.xlabel("Tiempo en sitio (min)")
plt.ylabel("Productos añadidos")
plt.show()
```



Interpretación de la gráfica: *Frontera de decisión – Regresión Logística*

La gráfica muestra cómo el modelo de regresión logística clasifica a los visitantes de una tienda online según:

- **Tiempo en sitio (minutos)**
- **Productos añadidos al carrito**

Y predice si **compran (1)** o **no compran (0)**.

1. Las dos zonas coloreadas del fondo

El fondo está dividido en dos regiones:

- **Zona azul** → el modelo predice **NO compra (0)**
- **Zona naranja** → el modelo predice **SÍ compra (1)**

Estas zonas representan la **frontera de decisión** aprendida por el modelo.

En tu caso, la frontera es **una línea diagonal**, lo cual es totalmente coherente porque:

- La regla que generó los datos es lineal:
tiempo > 5 y productos > 2 → compra = 1
 - La regresión logística aprende exactamente esa separación.
-

2. Los puntos del gráfico

Cada punto representa un visitante:

- **Azul** → no compró
- **Naranja** → sí compró

Lo interesante es que:

- Los puntos naranjas se concentran en la zona donde **tiempo es alto y productos es alto**.
- Los puntos azules se agrupan en el resto del espacio.

Esto confirma que el modelo está capturando correctamente el patrón de tus datos.

3. La frontera de decisión

La línea diagonal que separa ambas zonas es la frontera donde el modelo considera que la probabilidad de compra es **0.5**.

- Por encima de esa línea → predice compra
- Por debajo → predice no compra

La frontera es limpia y recta porque tus datos fueron generados con una regla muy clara y sin ruido.

Conclusión

La gráfica muestra que:

- El modelo separa muy bien los casos de compra y no compra.
- La frontera de decisión es lineal y coherente con la regla que generó los datos.
- La clasificación visual es clara y fácil de interpretar.

In []: